

SUPPORT

Starter Kit Frontend

React 18 + Vite + React Router DOM

Version avec TailwindCSS v4 (optionnel) + Layouts conditionnels

 **Objectif : Créer un frontend React connecté au backend avec authentification JWT**

1. Création du projet

On utilise Vite pour créer un projet React moderne et rapide.

1.1 Commandes d'installation

```
# Créer le projet React avec Vite
npm create vite@latest frontend -- --template react

# Entrer dans le dossier
cd frontend

# Installer les dépendances
npm install

# Ajouter React Router pour la navigation
npm install react-router-dom

# Lancer le serveur de développement
npm run dev
```

⚠ Le serveur démarre sur <http://localhost:5173> (pas 3000 comme Create React App)

1.2 Structure des dossiers à créer

```
mkdir -p src/components src/contexts src/hooks src/layouts src/pages src/services
```

Structure finale :

```
frontend/src/
├── components/      # Header, Footer, PrivateRoute
├── contexts/        # AuthContext (état global)
├── hooks/           # useAuth (accès simplifié)
├── layouts/         # MainLayout, AuthLayout
├── pages/           # Home, Login, Register, Dashboard
├── services/        # api.js (appels HTTP)
├── App.jsx          # Routes
├── main.jsx         # Point d'entrée
└── index.css        # Styles
```

2. Service API

Le service API centralise tous les appels HTTP vers le backend.

2.1 Fichier services/api.js

```
// services/api.js

const API_URL = 'http://localhost:5000/api';

async function fetchAPI(endpoint, options = {}) {
  const token = localStorage.getItem('token');

  const headers = {
    'Content-Type': 'application/json',
    ...(token && { Authorization: `Bearer ${token}` })
  };

  try {
    const response = await fetch(`${API_URL}${endpoint}`, {
      ...options,
      headers
    });

    const data = await response.json();

    if (!response.ok) {
      throw { status: response.status, message: data.error || 'Erreur' };
    }

    return data;
  } catch (error) {
    if (!error.status) {
      throw { status: 0, message: 'Serveur inaccessible' };
    }
    throw error;
  }
}

export const authService = {
  register: (userData) => fetchAPI('/auth/register', {
    method: 'POST',
    body: JSON.stringify(userData)
  }),

  login: (email, password) => fetchAPI('/auth/login', {
    method: 'POST',
    body: JSON.stringify({ email, password })
  }),

  getProfile: () => fetchAPI('/auth/me')
};
```

💡 `...(token && { Authorization: ... })` → Si token existe, ajoute le header. Sinon, n'ajoute rien.

3. Context d'Authentification

Le Context permet de partager l'utilisateur connecté dans toute l'app.

3.1 Fichier contexts/AuthContext.jsx

```
// contexts/AuthContext.jsx
import { createContext, useState, useEffect } from 'react';
import { authService } from '../services/api.js';

export const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const token = localStorage.getItem('token');
    if (token) {
      authService.getProfile()
        .then(data => setUser(data.user))
        .catch(() => localStorage.removeItem('token'))
        .finally(() => setLoading(false));
    } else {
      setLoading(false);
    }
  }, []);

  const login = async (email, password) => {
    const data = await authService.login(email, password);
    localStorage.setItem('token', data.token);
    setUser(data.user);
    return data;
  };

  const register = async (userData) => {
    const data = await authService.register(userData);
    localStorage.setItem('token', data.token);
    setUser(data.user);
    return data;
  };

  const logout = () => {
    localStorage.removeItem('token');
    setUser(null);
  };

  return (
    <AuthContext.Provider value={{
      user, loading, isAuthenticated: !!user,
      login, register, logout
    }}>
      {children}
    </AuthContext.Provider>
  );
}
```

4. Hook useAuth

4.1 Fichier hooks/useAuth.js

```
// hooks/useAuth.js
import { useContext } from 'react';
import { AuthContext } from '../contexts/AuthContext.jsx';

export function useAuth() {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error('useAuth doit être utilisé dans un AuthProvider');
  }
  return context;
}

export default useAuth;
```

4.2 Utilisation

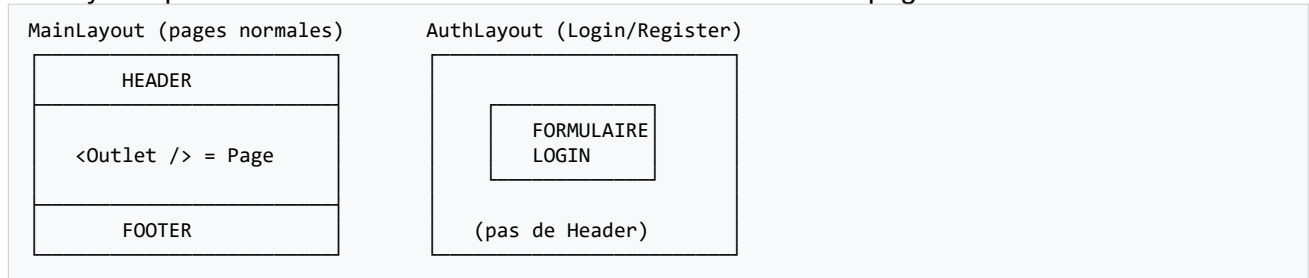
```
// Dans n'importe quel composant :
import { useAuth } from '../hooks/useAuth.js';

function MonComposant() {
  const { user, isAuthenticated, logout } = useAuth();

  return <p>Bonjour {user?.firstname} !</p>;
}
```

5. Layouts conditionnels

Les layouts permettent d'avoir des structures différentes selon les pages.



5.1 MainLayout (avec Header + Footer)

```
// layouts/MainLayout.jsx
import { Outlet } from 'react-router-dom';
import Header from '../components/Header.jsx';
import Footer from '../components/Footer.jsx';

function MainLayout() {
  return (
    <div>
      <Header />
      <main>
        <Outlet /> {/* ← La page enfant s'affiche ici */}
      </main>
      <Footer />
    </div>
  );
}

export default MainLayout;
```

5.2 AuthLayout (plein écran, pas de Header)

```
// layouts/AuthLayout.jsx
import { Outlet } from 'react-router-dom';

function AuthLayout() {
  return (
    <div>
      <Outlet />
    </div>
  );
}

export default AuthLayout;
```

💡 <Outlet /> affiche la page enfant correspondant à la route actuelle

6. Header et Footer

6.1 Header avec navigation conditionnelle

```
// components/Header.jsx
import { Link, NavLink, useNavigate } from 'react-router-dom';
import { useAuth } from '../hooks/useAuth.js';

function Header() {
  const { user, isAuthenticated, logout } = useAuth();
  const navigate = useNavigate();

  const handleLogout = () => {
    logout();
    navigate('/login');
  };

  return (
    <header>
      <Link to="/">Starter Kit</Link>

      <nav>
        <NavLink to="/">Accueil</NavLink>
        {isAuthenticated && <NavLink to="/dashboard">Dashboard</NavLink>}
      </nav>

      <div>
        {isAuthenticated ? (
          <>
            <span>{user?.firstname}</span>
            <button onClick={handleLogout}>Déconnexion</button>
          </>
        ) : (
          <>
            <Link to="/login">Connexion</Link>
            <Link to="/register">S'inscrire</Link>
          </>
        )}
      </div>
    </header>
  );
}

export default Header;
```

6.2 Footer

```
// components/Footer.jsx

function Footer() {
  return (
    <footer>
      <p>Starter Kit - © {new Date().getFullYear()} La Plateforme_</p>
    </footer>
  );
}

export default Footer;
```

7. Protection des routes

7.1 Composant PrivateRoute

```
// components/PrivateRoute.jsx
import { Navigate, useLocation } from 'react-router-dom';
import { useAuth } from '../hooks/useAuth.js';

function PrivateRoute({ children }) {
  const { isAuthenticated, loading } = useAuth();
  const location = useLocation();

  if (loading) {
    return <div><p>Chargement...</p></div>;
  }

  if (!isAuthenticated) {
    return <Navigate to="/login" state={{ from: location }} replace />;
  }

  return children;
}

export default PrivateRoute;
```

FLUX DE PROTECTION

```
Utilisateur non connecté veut accéder à /dashboard
↓
PrivateRoute vérifie
↓
isAuthenticated = false
↓
Redirection vers /login (avec mémorisation)
↓
Après connexion réussie
↓
Retour automatique à /dashboard
```


8. Page Login

```
// pages/Login.jsx
import { useState } from 'react';
import { Link, useNavigate, useLocation } from 'react-router-dom';
import { useAuth } from '../hooks/useAuth.js';

function Login() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);

  const { login } = useAuth();
  const navigate = useNavigate();
  const location = useLocation();
  const from = location.state?.from?.pathname || '/dashboard';

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');
    setLoading(true);

    try {
      await login(email, password);
      navigate(from, { replace: true });
    } catch (err) {
      setError(err.message || 'Erreur de connexion');
    } finally {
      setLoading(false);
    }
  };

  return (
    <div>
      <h1>Connexion</h1>
      {error && <p style={{color: 'red'}}>{error}</p>}

      <form onSubmit={handleSubmit}>
        <div>
          <label>Email</label>
          <input type="email" value={email}
            onChange={(e) => setEmail(e.target.value)} required />
        </div>
        <div>
          <label>Mot de passe</label>
          <input type="password" value={password}
            onChange={(e) => setPassword(e.target.value)} required />
        </div>
        <button type="submit" disabled={loading}>
          {loading ? 'Connexion...' : 'Se connecter'}
        </button>
      </form>

      <p>Pas de compte ? <Link to="/register">S'inscrire</Link></p>
    </div>
  );
}

export default Login;
```

8.1 Register

```
// pages/Register.jsx
import { useState } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { useAuth } from '../hooks/useAuth.js';

function Register() {
  const [formData, setFormData] = useState({
    firstname: '',
    lastname: '',
    email: '',
    password: '',
    confirmPassword: ''
  });
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);

  const { register } = useAuth();
  const navigate = useNavigate();

  const handleChange = (e) => {
    setFormData(prev => ({
      ...prev,
      [e.target.name]: e.target.value
    }));
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');

    // Vérifications
    if (formData.password !== formData.confirmPassword) {
      setError('Les mots de passe ne correspondent pas');
      return;
    }

    if (formData.password.length < 6) {
      setError('Le mot de passe doit contenir au moins 6 caractères');
      return;
    }

    setLoading(true);

    try {
      await register({
        firstname: formData.firstname,
        lastname: formData.lastname,
        email: formData.email,
        password: formData.password
      });
      navigate('/dashboard');
    } catch (err) {
      setError(err.message || 'Erreur lors de l\'inscription');
    } finally {
      setLoading(false);
    }
  };

  return (
    <div>
      <h1>Créer un compte</h1>

      {error && <p style={{color: 'red'}}>{error}</p>}
    </div>
  );
}
```

```

<form onSubmit={handleSubmit}>
  <div>
    <label>Prénom</label>
    <input
      type="text"
      name="firstname"
      value={formData.firstname}
      onChange={handleChange}
      required
      disabled={loading}
    />
  </div>

  <div>
    <label>Nom</label>
    <input
      type="text"
      name="lastname"
      value={formData.lastname}
      onChange={handleChange}
      required
      disabled={loading}
    />
  </div>

  <div>
    <label>Email</label>
    <input
      type="email"
      name="email"
      value={formData.email}
      onChange={handleChange}
      required
      disabled={loading}
    />
  </div>

  <div>
    <label>Mot de passe</label>
    <input
      type="password"
      name="password"
      value={formData.password}
      onChange={handleChange}
      required
      disabled={loading}
    />
  </div>

  <div>
    <label>Confirmer le mot de passe</label>
    <input
      type="password"
      name="confirmPassword"
      value={formData.confirmPassword}
      onChange={handleChange}
      required
      disabled={loading}
    />
  </div>

  <button type="submit" disabled={loading}>
    {loading ? 'Inscription...' : 'S\'inscrire'}
  </button>
</form>

```

```
    <p>
      Déjà un compte ? <Link to="/login">Se connecter</Link>
    </p>
  </div>
);
}

export default Register;
```

8.2 Dashborad

```
import { useAuth } from '../hooks/useAuth.js';

function Dashboard() {
  const { user } = useAuth();

  return (
    <div>
      <h1>Bienvenue {user?.firstname} !</h1>
      <p>Email : {user?.email}</p>
    </div>
  );
}

export default Dashboard;
```

8.2 Home

```
// pages/Home.jsx
import { Link } from 'react-router-dom';
import { useAuth } from '../hooks/useAuth.js';

function Home() {
  const { isAuthenticated } = useAuth();

  return (
    <div>
      <h1>Bienvenue sur le Starter Kit</h1>
      <p>Template moderne React + Node.js + MySQL</p>

      <div>
        {isAuthenticated ? (
          <Link to="/dashboard">Accéder au Dashboard</Link>
        ) : (
          <>
            <Link to="/register">Commencer</Link>
            <Link to="/login">Se connecter</Link>
          </>
        )}
      </div>
    </div>
  );
}

export default Home;
```

9. Configuration des Routes

9.1 App.jsx avec layouts imbriqués

```
// App.jsx
import { Routes, Route, Navigate } from 'react-router-dom';
import { useAuth } from './hooks/useAuth.js';

import MainLayout from './layouts/MainLayout.jsx';
import AuthLayout from './layouts/AuthLayout.jsx';
import PrivateRoute from './components/PrivateRoute.jsx';

import Home from './pages/Home.jsx';
import Login from './pages/Login.jsx';
import Register from './pages/Register.jsx';
import Dashboard from './pages/Dashboard.jsx';

function App() {
  const { loading } = useAuth();

  if (loading) return <div><p>Chargement...</p></div>;

  return (
    <Routes>
      {/* Routes AVEC Header + Footer */}
      <Route element={<MainLayout />}>
        <Route path="/" element={<Home /> } />
        <Route path="/dashboard" element={
          <PrivateRoute><Dashboard /></PrivateRoute>
        } />
      </Route>

      {/* Routes SANS Header (plein écran) */}
      <Route element={<AuthLayout />}>
        <Route path="/login" element={<Login /> } />
        <Route path="/register" element={<Register /> } />
      </Route>

      <Route path="*" element={<Navigate to="/" /> } />
    </Routes>
  );
}

export default App;
```

9.2 main.jsx avec providers

```
// main.jsx
import React from 'react';
import ReactDOM from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import App from './App.jsx';
import { AuthProvider } from './contexts/AuthContext.jsx';
import './index.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <BrowserRouter>
      <AuthProvider>
        <App />
      </AuthProvider>
    </BrowserRouter>
  </React.StrictMode>
);
```

⚠ L'ordre est important : BrowserRouter → AuthProvider → App

10. TailwindCSS v4 (optionnel)

10.1 Installation simplifiée

```
npm install tailwindcss @tailwindcss/vite
```

NE PAS FAIRE (c'était v3) :

```
npm install -D postcss autoprefixer  
npx tailwindcss init -p
```

10.2 vite.config.js

```
import { defineConfig } from 'vite'  
import react from '@vitejs/plugin-react'  
import tailwindcss from '@tailwindcss/vite'  
  
export default defineConfig({  
  plugins: [react(), tailwindcss()],  
})
```

10.3 index.css

```
/* 1 seule ligne ! */  
@import "tailwindcss";  
  
/* Classes réutilisables (optionnel) */  
@layer components {  
  .btn { @apply px-6 py-2 rounded-lg font-medium transition-all; }  
  .btn-primary { @apply bg-indigo-600 text-white hover:bg-indigo-700; }  
  .form-input { @apply w-full px-4 py-2 border-2 rounded-lg; }  
}
```

11. Récapitulatif

11.1 Ordre de création

#	Fichier	Description
1	services/api.js	Appels API
2	contexts/AuthContext.jsx	État global
3	hooks/useAuth.js	Accès au contexte
4	components/Header.jsx	Navigation
5	components/Footer.jsx	Pied de page
6	components/PrivateRoute.jsx	Protection
7	layouts/MainLayout.jsx	Structure normale
8	layouts/AuthLayout.jsx	Structure auth
9	pages/*.jsx	Les pages
10	App.jsx	Routes
11	main.jsx	Point d'entrée

11.2 Checklist

- ☐ npm install + react-router-dom
- ☐ Structure dossiers créée
- ☐ services/api.js
- ☐ AuthContext + AuthProvider
- ☐ Hook useAuth
- ☐ Header + Footer
- ☐ MainLayout + AuthLayout
- ☐ PrivateRoute
- ☐ Pages (Home, Login, Register, Dashboard)
- ☐ App.jsx (routes)
- ☐ main.jsx (providers)

Formation DWWM - La Plateforme_